Devsinc Enhanced Secure Coding Practices

Purpose & Scope

This guide establishes security standards for all software development at Devsinc, applicable across all tech stacks and service models (B2B/B2C). These practices are based on OWASP guidelines and industry best practices, designed to protect client confidentiality, data integrity, and system availability.

Target Audience: All developers, tech leads, and engineering managers
Mandatory Compliance: All production code must adhere to these standards
Last Updated: November 2025

---

1. Input Validation & Sanitization

Core Principles

- **Trust nothing from the client** - All external input is potentially malicious
- **Validate on the server** - Never rely solely on client-side validation
- **Fail securely** - Reject invalid input, don't try to "clean" it

Implementation Requirements

✅ DO

```
✓ Validate ALL user inputs (forms, APIs, URLs, headers, cookies)
✓ Use allowlists (whitelist) over denylists (blacklist)
✓ Validate data type, length, format, and range
✓ Implement centralized validation functions/middleware
✓ Sanitize inputs before database queries and output rendering
```

❌ DON'T

```
✗ Trust data just because it came from your own API
✗ Use blocklists alone (blacklists are incomplete)
✗ Allow unvalidated redirects or file paths
✗ Concatenate user input directly into SQL/commands
```

Dangerous Characters to Handle

```
< > " ' % ( ) & + \ \' \" ; | * ? ~ < > ^ [ ] { } $ \n \r %00
```

```
../ ..\ %0d %0a null bytes
```

Recommended Approach

Use validation libraries specific to your framework rather than writing custom validation logic. Most modern frameworks provide built-in validators for common data types like emails, URLs, numbers, and dates. Leverage these tools to ensure consistent validation across your application.

---

2. Authentication & Password Management

Core Requirements

Password Policies

- **Minimum Length:** 12 characters (16+ recommended)
- **Complexity:** Mix of uppercase, lowercase, numbers, and special characters
- **History:** Prevent reuse of last 5 passwords
- **Expiration:** 90 days for critical systems (if required by client)

Account Security

- **Failed Login Attempts:** Lock account after 5 failed attempts
- **Lockout Duration:** 15-30 minutes (or require admin unlock)
- **Session Timeout:** 15 minutes inactivity for sensitive apps, 2 hours for standard
- **MFA:** Mandatory for admin accounts and sensitive operations

Implementation Standards

✅ **DO**

```
✓ Hash passwords using bcrypt, Argon2, or PBKDF2
✓ Use salt with minimum 32 random characters
✓ Store authentication credentials on server only
✓ Implement centralized authentication service
✓ Use generic error messages ("Invalid credentials")
✓ Enforce password changes on first login with temp passwords
✓ Require re-authentication for sensitive operations
```

❌ **DON'T**

✗ Store passwords in plain text or with weak encryption
✗ Use MD5 or SHA-1 for password hashing
✗ Reveal which part failed (username vs password)
✗ Allow default vendor credentials in production
✗ Store credentials in source code or config files (use env variables)

Recommended Approach

Use industry-standard hashing libraries available in your framework. Modern frameworks provide built-in password hashing with appropriate salt and iteration counts. For Node.js use bcrypt, for Python use Django's built-in hashers or passlib, for PHP use password_hash with PASSWORD_ARGON2ID or PASSWORD_BCRYPT. Never implement your own hashing algorithm.

For rate limiting, implement middleware or use your API gateway's built-in features to track and limit authentication attempts per IP address or username.

---

3. Session Management

Core Requirements

✅ DO

✓ Use framework's built-in session management
✓ Generate new session ID after login/privilege change
✓ Set appropriate cookie flags (HttpOnly, Secure, SameSite)
✓ Implement absolute and idle timeouts
✓ Destroy sessions completely on logout
✓ Store session data server-side only

❌ DON'T

✗ Expose session IDs in URLs or logs
✗ Use predictable session IDs
✗ Store sensitive data in client-side storage
✗ Allow concurrent sessions (if not required)

Recommended Approach

Use your framework's built-in session management rather than implementing custom solutions. Configure session cookies with these essential flags: HttpOnly (prevents JavaScript access), Secure (HTTPS only), and SameSite (CSRF protection). Set appropriate timeout values based on your application's security requirements - shorter for sensitive applications, reasonable for standard use cases.

---

4. Access Control & Authorization

Core Principles

- **Principle of Least Privilege:** Grant minimum necessary permissions
- **Defense in Depth:** Multiple layers of access control
- **Deny by Default:** Explicitly allow, implicitly deny

✅ DO

```
✓ Verify authorization on EVERY request
✓ Implement role-based access control (RBAC)
✓ Use centralized authorization logic
✓ Check permissions server-side
✓ Separate privileged functions from regular code
✓ Enforce resource-level permissions
✓ Audit access to sensitive operations
```

❌ DON'T

```
✗ Rely on client-side authorization checks
✗ Use insecure direct object references
✗ Expose internal IDs or GUIDs directly
✗ Skip authorization on "internal" APIs
```

Recommended Approach

Create middleware or decorators that verify user permissions before executing any protected functionality. Keep authorization logic centralized and reusable across your application. Always perform these checks on the server side, regardless of any client-side restrictions you may have implemented.

---

5. Cryptography & Data Protection

Core Requirements

Encryption Standards

- **Algorithm:** AES-256 for symmetric, RSA-2048+ for asymmetric
- **TLS:** Use TLS 1.2 minimum (TLS 1.3 preferred)

- **Random Numbers:** Use cryptographically secure generators
- **Key Storage:** Use key management services (AWS KMS, Azure Key Vault)

✅ DO

```
✓ Encrypt all sensitive data at rest and in transit
✓ Use proven cryptographic libraries (don't roll your own)
✓ Protect encryption keys separately from data
✓ Use environment variables for secrets, never hardcode
✓ Implement secure key rotation policies
✓ Hash passwords with salt using bcrypt/Argon2
```

❌ DON'T

```
✗ Store secrets in source code, config files, or databases
✗ Use weak algorithms (DES, MD5, SHA1)
✗ Implement custom encryption algorithms
✗ Reuse initialization vectors (IVs)
✗ Store keys alongside encrypted data
```

Recommended Approach

Use your programming language's built-in cryptographic libraries. For random number generation, use cryptographically secure random generators (crypto.randomBytes in Node.js, secrets module in Python, random_bytes in PHP). Store encryption keys in environment variables or dedicated key management services like AWS KMS or Azure Key Vault, never in your codebase or database alongside encrypted data.

---

6. Error Handling & Logging

Core Requirements

✅ DO

```
✓ Log all security events (auth failures, access denials, etc.)
✓ Use generic error messages for users
✓ Log detailed errors server-side only
✓ Implement centralized logging
✓ Protect logs with access controls
✓ Set up log monitoring and alerts
✓ Include timestamps, user IDs, and IP addresses
```

❌ **DON'T**

```
✗ Expose stack traces to users
✗ Log passwords, tokens, or sensitive data
✗ Display database errors to users
✗ Include system paths in error messages
✗ Log full request bodies with sensitive data
```

What to Log

Security Events to Always Log:

- All authentication attempts (success/failure)
- Authorization failures
- Input validation failures
- Session management events
- Security configuration changes
- Suspicious patterns (e.g., rate limit violations)

Implementation Guidelines: Use structured logging with consistent formats. Include request context (timestamp, user ID, IP address, request ID) without logging sensitive data like passwords, tokens, or full request bodies containing personal information.

---

7. Database Security

Core Requirements

✅ **DO**

```
✓ Use parameterized queries/prepared statements ALWAYS
✓ Use ORM query builders properly
✓ Implement least privilege for database users
✓ Encrypt database connections (SSL/TLS)
✓ Store connection strings in environment variables
✓ Close connections promptly
✓ Regularly backup and test restoration
✓ Keep database software patched
```

❌ **DON'T**

✗ Concatenate user input into SQL queries
✗ Use database root/admin accounts for applications
✗ Hardcode connection strings
✗ Expose database errors to users
✗ Grant unnecessary database permissions

Recommended Approach

Always use parameterized queries or your ORM's query builder methods. These automatically handle escaping and prevent SQL injection. Never concatenate user input directly into SQL strings, even if you think you've sanitized it. Configure your database connection with the minimum privileges needed - your application should not connect as a database administrator.

---

8. API Security

Core Requirements

✅ DO

✓ Authenticate all API endpoints (except public ones)
✓ Use API keys/tokens with proper scoping
✓ Implement rate limiting (per user/IP)
✓ Validate request content types
✓ Use HTTPS only
✓ Implement CORS properly
✓ Version your APIs
✓ Document security requirements in API specs

❌ DON'T

✗ Expose internal API endpoints publicly
✗ Use GET for state-changing operations
✗ Return excessive data (limit fields)
✗ Expose system information in responses
✗ Allow unrestricted file uploads

Recommended Approach

Implement rate limiting at multiple levels: per IP address, per user account, and per API endpoint based on sensitivity. Use existing rate limiting middleware or API gateway features rather than building custom solutions. For public APIs, consider implementing tiered rate limits based on authentication status or subscription level.

9. Frontend Security

Core Requirements

✅ DO

```
✓ Implement Content Security Policy (CSP)
✓ Use Subresource Integrity (SRI) for CDN resources
✓ Sanitize all user-generated content before rendering
✓ Set secure HTTP headers
✓ Validate and sanitize URLs before navigation
✓ Use HTTPS everywhere
✓ Implement CSRF tokens for state-changing operations
```

❌ DON'T

```
✗ Use eval() or innerHTML with user data
✗ Store sensitive data in localStorage/sessionStorage
✗ Include sensitive data in URLs
✗ Disable security features for "convenience"
✗ Trust data from URL parameters
```

Recommended Approach

Use your framework's built-in HTML escaping and sanitization features. Modern frameworks like React, Vue, and Angular escape content by default. When you must render user-generated HTML, use a trusted sanitization library like DOMPurify. Implement Content Security Policy headers to provide an additional layer of defense against XSS attacks.

For CSRF protection, use your framework's built-in CSRF token mechanism. These typically work by embedding a secret token in forms and validating it on the server for state-changing operations.

---

10. File Upload Security

Core Requirements

✅ DO

✓ Require authentication for uploads
✓ Validate file types using MIME type and extension
✓ Scan files for malware
✓ Limit file size (define max per file type)
✓ Store files outside web root
✓ Generate random filenames
✓ Implement virus scanning for uploads
✓ Use content delivery network (CDN) for serving

❌ DON'T

✗ Trust client-provided filenames
✗ Allow executable file uploads
✗ Store uploads in publicly accessible directories
✗ Rely solely on extension validation
✗ Allow unrestricted file sizes

Recommended Approach

Validate file uploads using multiple checks: file extension, MIME type, and actual file content when possible. Store uploaded files outside your web application's document root and generate random filenames to prevent direct access and path traversal attacks. Implement virus scanning for all uploads if your application handles files from untrusted sources. Use a separate storage service or CDN for serving user-uploaded files with appropriate access controls.

---

11. Dependency Management

Core Requirements

✅ DO

✓ Keep all dependencies up to date
✓ Audit dependencies regularly (npm audit, pip-audit, etc.)
✓ Use dependency lock files (package-lock.json, Pipfile.lock)
✓ Review third-party packages before adding
✓ Monitor security advisories
✓ Remove unused dependencies
✓ Use automated tools (Dependabot, Snyk, etc.)

❌ DON'T

✗ Ignore security warnings
✗ Use dependencies with known vulnerabilities
✗ Install packages without verification
✗ Use outdated or unmaintained libraries

Dependency Scanning Commands

Run these commands regularly in your projects to identify vulnerable dependencies:

Node.js: `npm audit` or `npm audit fix`
Python: `pip-audit` or `safety check`
PHP: `composer audit`
Java: `mvn dependency:tree` followed by vulnerability scanning tools
.NET: `dotnet list package --vulnerable`

Integrate these checks into your CI/CD pipeline to catch vulnerabilities before deployment.

---

12. Code Quality & Testing

Core Requirements

✅ DO

✓ Use linters and formatters (ESLint, Prettier, Black, etc.)
✓ Write security test cases
✓ Perform code reviews for all changes
✓ Use static analysis tools (SonarQube, CodeQL)
✓ Implement automated security testing in CI/CD
✓ Follow language-specific best practices
✓ Document security-critical code

Security Testing Checklist

- [ ] Unit tests for authentication/authorization
- [ ] Input validation tests with malicious payloads
- [ ] SQL injection tests
- [ ] XSS payload tests
- [ ] CSRF token validation tests
- [ ] Rate limiting tests
- [ ] Session management tests
- [ ] File upload security tests

---

13. Configuration & Deployment

Core Requirements

✅ DO

```
✓ Use environment-specific configurations
✓ Store secrets in secure vaults (AWS Secrets Manager, etc.)
✓ Remove debug/verbose logging in production
✓ Disable directory listing
✓ Remove default accounts and pages
✓ Use infrastructure as code (IaC)
✓ Implement automated deployment pipelines
✓ Maintain separate environments (dev, staging, prod)
```

❌ DON'T

```
✗ Commit secrets to version control
✗ Use default credentials
✗ Deploy with debug mode enabled
✗ Expose .git, .env, or config files publicly
✗ Run services as root/admin
```

Environment Variables Best Practices

Use environment variables for all configuration that varies between environments or contains sensitive data. Never commit actual secrets to version control. Instead, commit a template file (like .env.example) that shows what variables are needed without exposing real values. Use a secrets management service in production rather than plain text environment files.

Example .env.example (safe to commit):

```
DATABASE_URL=postgresql://localhost/mydb
API_KEY=your_api_key_here
SESSION_SECRET=generate_random_secret
JWT_SECRET=generate_random_jwt_secret
```

The actual .env file with real credentials should be in your .gitignore and never committed to version control.

---

14. Incident Response

Security Incident Procedure

1. **Detect & Report:** Immediately report to security team
2. **Contain:** Isolate affected systems
3. **Investigate:** Analyze logs, determine scope
4. **Remediate:** Apply fixes, patch vulnerabilities
5. **Recover:** Restore services securely
6. **Post-Mortem:** Document lessons learned

Contact Information

- **Security Team:** security@devsinc.com
- **Engineering Manager:** [Your contact]
- **Emergency Hotline:** [Contact number]

---

15. Compliance & Responsibilities

Developer Responsibilities

- Follow these guidelines in all code
- Complete security training annually
- Report vulnerabilities immediately
- Participate in security code reviews
- Keep knowledge updated

Manager Responsibilities

- Ensure team compliance
- Allocate time for security updates
- Review security metrics
- Support security initiatives
- Escalate issues appropriately

Consequences of Non-Compliance

- Code rejection in reviews
- Delayed deployment
- Escalation to management
- Required remediation training

Resources & Tools

Recommended Tools by Category

Static Analysis:

- SonarQube, ESLint, Bandit, Brakeman, SpotBugs

Dependency Scanning:

- Snyk, Dependabot, OWASP Dependency-Check

DAST (Dynamic Testing):

- OWASP ZAP, Burp Suite

Secret Scanning:

- GitGuardian, TruffleHog, detect-secrets

Security Headers:

- Helmet.js (Node), Security Headers Analyzer

Learning Resources

- OWASP Top 10: https://owasp.org/www-project-top-ten/
- OWASP Cheat Sheets: https://cheatsheetseries.owasp.org/
- CWE Top 25: https://cwe.mitre.org/top25/

Implementation Timeline

Phase 1 (Immediate - Week 1-2)

- [ ] Set up dependency scanning
- [ ] Enable authentication on all non-public endpoints

- [ ] Implement rate limiting on APIs
- [ ] Add security headers

Phase 2 (Short-term - Month 1)

- [ ] Audit and update all dependencies
- [ ] Implement centralized logging
- [ ] Add input validation middleware
- [ ] Set up automated security scanning in CI/CD

Phase 3 (Medium-term - Quarter 1)

- [ ] Complete security training for all developers
- [ ] Implement comprehensive testing
- [ ] Conduct security code review of critical modules
- [ ] Establish security metrics and monitoring

Review & Updates

- **Review Frequency:** Quarterly
- **Update Process:** Security team proposes, leadership approves
- **Feedback:** Submit suggestions to security@devsinc.com
- **Version:** 2.0
- **Last Updated:** November 2025
- **Next Review:** February 2026

Appendix: Quick Reference Checklists

Pre-Deployment Security Checklist

- [ ] All dependencies updated and scanned
- [ ] No secrets in code or config files
- [ ] Input validation on all endpoints
- [ ] Authentication/authorization properly implemented
- [ ] HTTPS enforced
- [ ] Security headers configured
- [ ] Error handling doesn't leak information
- [ ] Logging configured and tested

- [ ] Database connections use parameterized queries
- [ ] Rate limiting enabled
- [ ] File uploads validated and secured
- [ ] CORS configured appropriately
- [ ] Security tests passing

Code Review Security Checklist

- [ ] No SQL injection vulnerabilities
- [ ] No XSS vulnerabilities
- [ ] Authentication properly implemented
- [ ] Authorization checked on all protected resources
- [ ] Sensitive data encrypted
- [ ] Passwords hashed with bcrypt/Argon2
- [ ] Input validation present
- [ ] Error messages don't leak info
- [ ] No hardcoded secrets
- [ ] Dependencies are up to date

Questions or clarifications? Contact your Engineering Manager or the Security Team.